AI-ASSISTED CODING

ASSIGNMENT -8.3

NAME: V.siddartha
Hall:2303A52062
Batch:45

#question 1

Task 1: Email Validation using TDD

Scenario

You are developing a user registration system that requires reliable email input validation.

Requirements

- Must contain @ and . characters
- Must not start or end with special characters
- Should not allow multiple @ symbols
- AI should generate test cases covering valid and invalid email formats
- Implement is_valid_email(email) to pass all AI-generated test cases

Expected Output

- Python function for email validation
- All AI-generated test cases pass successfully
- Invalid email formats are correctly rejected
- Valid email formats return True

#prompt and code

```
#create a email Validation using TDD it must contain @ and .com with out any special character at start and end of the email address
import re
def validate_email(email):
    # Regular expression pattern for validating email
    pattern = r'^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$'

    # Check if the email matches the pattern
    if re.match(pattern, email):
        return True
    else:
        return False
#allow lower case and don't allow upper case letters in the email address
def validate_email(email):
    # Regular expression pattern for validating email
    pattern = r'^[a-z0-9._%+-]+@[a-z0-9.-]+\.[a-z]{2,}$'

    # Check if the email matches the pattern
```

```
    if re.match(pattern, email):
        return True
    else:
        return False
#dont allow special characters at the start and end of the email address
def validate_email(email):
    # Regular expression pattern for validating email
    pattern = r'^[a-z0-9]+[a-z0-9._%+-]*@[a-z0-9.-]+\.[a-z]{2,}$'

    # Check if the email matches the pattern
    if re.match(pattern, email):
        return True
    else:
        return False
#dont allow multiple @ in the email address and multiple .com in the email address
def validate_email(email):
    # Regular expression pattern for validating email
    pattern = r'^[a-z0-9]+[a-z0-9._%+-]*@[a-z0-9.-]+\.[a-z]{2,}$'

    # Check if the email matches the pattern
    if re.match(pattern, email) and email.count('@') == 1 and email.count('.com') <= 1:
        return True
    else:
        return False
#accept user input and validate the email address
def validate_email(email):
    # Regular expression pattern for validating email
    pattern = r'^[a-z0-9]+[a-z0-9._%+-]*@[a-z0-9.-]+\.[a-z]{2,}$'

    # Check if the email matches the pattern
    if re.match(pattern, email) and email.count('@') == 1 and email.count('.com') <= 1:
        return True
    else:
        return False
email = input("Enter your email address: ")
if validate_email(email):
    print("Valid email address.")
else:
    print("Invalid email address.")
```

#OUTPUT

```
PS C:\Users\sidda\OneDrive\Documents\Desktop\AI - Assisted coding> & C:\Users\sidda\AppData\Local\Programs\Python\Pyth
on314\python.exe "c:/Users/sidda/OneDrive/Documents/Desktop/AI - Assisted coding/assisnment 8.3.py"
Enter your email address: Siddarthavelpula87@gmail.com
Invalid email address.
PS C:\Users\sidda\OneDrive\Documents\Desktop\AI - Assisted coding> & C:\Users\sidda\AppData\Local\Programs\Python\Pyth
on314\python.exe "c:/Users/sidda/OneDrive/Documents/Desktop/AI - Assisted coding/assisnment 8.3.py"
Enter your email address: siddarthavalpula87@gmail.com
Valid email address.
```

#question 2

Task 2: Grade Assignment using Loops

Scenario

You are building an automated grading system for an online examination platform.

Requirements

• AI should generate test cases for assign_grade(score) where:

– 90–100 → A

– 80–89 → B

– 70–79 → C

– 60–69 → D

– Below 60 → F

• Include boundary values (60, 70, 80, 90)

• Include invalid inputs such as -5, 105, "eighty"

• Implement the function using a test-driven approach

Expected Output

• Grade assignment function implemented in Python

• Boundary values handled correctly

• Invalid inputs handled gracefully

• All AI-generated test cases pass

#prompt&code

```
#TASK 8.3 2
#build an online grading system for an online examination the grading should be based on the following criteria:
#90-100: A
#80-89: B
#70-79: C
#60-69: D
#below 60: F
#include boumdary values in the grading system
def grade(score):
    if 90 <= score <= 100:
        return 'A'
```

```
    elif 80 <= score < 90:
        return 'B'
    elif 70 <= score < 80:
        return 'C'
    elif 60 <= score < 70:
        return 'D'
    elif score < 60:
        return 'F'
    else:
        return 'Invalid score'
# print invalid for -ve values and values above 100 and also for non numeric values and print
valid for values between 0 and 100
def grade(score):
    if not isinstance(score, (int, float)):
        return 'Invalid score'
    if score < 0 or score > 100:
        return 'Invalid score'
    if 90 <= score <= 100:
        return 'A'
    elif 80 <= score < 90:
        return 'B'
    elif 70 <= score < 80:
        return 'C'
    elif 60 <= score < 70:
        return 'D'
    else:
        return 'F'
#accept user input and validate the score and print the grade
def grade(score):
    if not isinstance(score, (int, float)):
        return 'Invalid score'
    if score < 0 or score > 100:
        return 'Invalid score'
    if 90 <= score <= 100:
        return 'A'
    elif 80 <= score < 90:
        return 'B'
    elif 70 <= score < 80:
        return 'C'
    elif 60 <= score < 70:
        return 'D'
```

```
    else:
        return 'F'
score = input("Enter your score: ")
try:
    score = float(score)
    print("Your grade is:", grade(score))
except ValueError:
    print("Invalid input. Please enter a numeric value for the score.")
```

#OUTPUT

```
PS C:\Users\sidda\OneDrive\Documents\Desktop\AI - Assisted coding> & C:\Users\sidda\AppData\Local\Programs\Python\Pyth
on314\python.exe "c:/Users/sidda/OneDrive/Documents/Desktop/AI - Assisted coding/assisnment 8.3-2.py"
Enter your score: 12
Your grade is: F
PS C:\Users\sidda\OneDrive\Documents\Desktop\AI - Assisted coding> & C:\Users\sidda\AppData\Local\Programs\Python\Pyth
on314\python.exe "c:/Users/sidda/OneDrive/Documents/Desktop/AI - Assisted coding/assisnment 8.3-2.py"
Enter your score: 100
Your grade is: A
PS C:\Users\sidda\OneDrive\Documents\Desktop\AI - Assisted coding> & C:\Users\sidda\AppData\Local\Programs\Python\Pyth
on314\python.exe "c:/Users/sidda/OneDrive/Documents/Desktop/AI - Assisted coding/assisnment 8.3-2.py"
Enter your score: 60
Your grade is: D
PS C:\Users\sidda\OneDrive\Documents\Desktop\AI - Assisted coding>
```

#QUESTION-3

Task 3: Sentence Palindrome Checker

Scenario

You are developing a text-processing utility to analyze sentences.

Requirements

- **AI should generate test cases for is_sentence_palindrome(sentence)**
- **Ignore case, spaces, and punctuation**
- **Test both palindromic and non-palindromic sentences**
- **Example:**

– "A man a plan a canal Panama" → True

Expected Output

- **Function correctly identifies sentence palindromes**
- **Case and punctuation are ignored**
- **Returns True or False accurately**
- **All AI-generated test cases pass**

#PROMPT AND CODE

genrate a sentence plaindrome checker that checks if a given sentence is a palindrome or not. A palindrome is a word, phrase, number, or other sequence of characters that reads the same forward and backward (ignoring spaces, punctuation, and capitalization).

```
def is_palindrome(sentence):
    # Remove spaces, punctuation, and convert to lowercase
    cleaned_sentence = ''.join(char.lower() for char in sentence if char.isalnum())

    # Check if the cleaned sentence is equal to its reverse
    return cleaned_sentence == cleaned_sentence[::-1]
# Accept user input and check if it's a palindrome
user_input = input("Enter a sentence: ")
if is_palindrome(user_input):
    print("The sentence is a palindrome.")
else:
    print("The sentence is not a palindrome.")
```

#OUTPUT

```
PS C:\Users\sidda\OneDrive\Documents\Desktop\AI - Assisted coding> & C:\Users\sidda\AppData\Local\Programs\Python\Pyth
on314\python.exe "c:/Users/sidda/OneDrive/Documents/Desktop/AI - Assisted coding/assisnment 8.3-3.py"
Enter a sentence: SYS
The sentence is a palindrome.
PS C:\Users\sidda\OneDrive\Documents\Desktop\AI - Assisted coding> & C:\Users\sidda\AppData\Local\Programs\Python\Pyth
on314\python.exe "c:/Users/sidda/OneDrive/Documents/Desktop/AI - Assisted coding/assisnment 8.3-3.py"
Enter a sentence: SIA
The sentence is not a palindrome.
PS C:\Users\sidda\OneDrive\Documents\Desktop\AI - Assisted coding>
```

#QUESTION -4

Task 4: ShoppingCart Class

Scenario

You are designing a basic shopping cart module for an e-commerce application.

Requirements

- AI should generate test cases for the ShoppingCart class
- Class must include the following methods:

– add_item(name, price)

– remove_item(name)

– total_cost()

- Validate correct addition, removal, and cost calculation
- Handle empty cart scenarios

Expected Output

- Fully implemented ShoppingCart class

• All methods pass AI-generated test cases
• Total cost is calculated accurately
• Items are added and removed correctly

#PROMPT AND CODE

```
# genrate a basic shopping cart module for an e-commerce application.
# Class must include the following methods:
# add_item(name, price)
# remove_item(name)
# total_cost()
# Validate correct addition, removal, and cost calculation
# Handle empty cart scenarios
class ShoppingCart:
    def __init__(self):
        self.cart = {}

    def add_item(self, name, price):
        if name in self.cart:
            self.cart[name] += price
        else:
            self.cart[name] = price

    def remove_item(self, name):
        if name in self.cart:
            del self.cart[name]
        else:
            print(f"Item '{name}' not found in the cart.")

    def total_cost(self):
        return sum(self.cart.values())
# Example usage
cart = ShoppingCart()
cart.add_item("Laptop", 999.99)
cart.add_item("Headphones", 199.99)
print("Total cost:", cart.total_cost())  # Total cost: 1199.98
cart.remove_item("Headphones")
print("Total cost after removal:", cart.total_cost())  # Total cost after removal: 999.99
cart.remove_item("Smartphone")  # Item 'Smartphone' not found in the cart
```

#OUTPUT

```
PS C:\Users\sidda\OneDrive\Documents\Desktop\AI - Assisted coding> & C:\Users\sidda\AppData\Local\Programs\Python\Pyth
on314\python.exe "c:/Users/sidda/OneDrive/Documents/Desktop/AI - Assisted coding/assisnment 8.3-4.py"
Total cost: 1199.98
Total cost after removal: 999.99
Item 'Smartphone' not found in the cart.
PS C:\Users\sidda\OneDrive\Documents\Desktop\AI - Assisted coding>
```

#QUESTION-5

Task 5: Date Format Conversion

Scenario

You are creating a utility function to convert date formats for reports.

Requirements

• AI should generate test cases for convert_date_format(date_str)

• Input format must be "YYYY-MM-DD"

• Output format must be "DD-MM-YYYY"

• Example:

– "2023-10-15" → "15-10-2023"

Expected Output

• Date conversion function implemented in Python

• Correct format conversion for all valid inputs

• All AI-generated test cases pass successfully

#PROMPT AND CODE

```
# create a a utility function to convert date formats for reports. The function should take a date string in the format "DD/MM/YYYY" and convert it to "YYYY-MM-DD".
def convert_date_format(date_str):
    try:
        day, month, year = date_str.split('/')
        return f"{year}-{month.zfill(2)}-{day.zfill(2)}"
    except ValueError:
        return "Invalid date format. Please use 'DD/MM/YYYY'."
# Example usage
date_input = "25/12/2021"
converted_date = convert_date_format(date_input)
print("Converted date:", converted_date)  # Converted date: 2021-12-25
```

#OUTPUT

```
PS C:\Users\sidda\OneDrive\Documents\Desktop\AI - Assisted coding> & C:\Users\sidda\AppData\Local\Programs\Python\Pyth
on314\python.exe "c:/Users/sidda/OneDrive/Documents/Desktop/AI - Assisted coding/assisnment 8.3-4.py"
Converted date: 2021-12-25
PS C:\Users\sidda\OneDrive\Documents\Desktop\AI - Assisted coding>
```